

Software Design Document

Complexity Estimating Remote Code Execution Engine

Author(s):	Jade N.
Course / Context:	Personal / independent project
Version:	0.1
Date:	September 8, 2026

*The template for this design document (without any project specific details) was generated by
a Qwen 3.5 Model*

Revision History

Version	Date	Author	Description
0.1	August 10, 2026	Jade N.	Initial draft
0.2	September 8, 2026	Jade N.	Feedback Update

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Tools and libraries	3
1.4	Definitions and Acronyms	4
1.5	References	4
2	System Overview	4
2.1	User Experience	4
2.2	Goals and Non-Goals	4
2.3	Assumptions and Constraints	4
3	Architecture	5
3.1	High-Level Architecture	5
3.2	Component Descriptions	5
3.2.1	Frontend	5
3.2.2	Core Logic	5
3.2.3	Sandboxed Container	5
3.2.4	General Complexity Analysis	5
3.2.5	Static Analysis	6
3.2.6	Dynamic Analysis	6
4	Detailed Design	6
4.1	Module: Core Logic	6
4.2	Module: General Complexity Analysis	8
4.3	Module: Dynamic Complexity Analysis	9
4.4	Module: Static Complexity Analysis	12
5	Testing Strategy	15
6	Risks and Open Questions	16

1 Introduction

1.1 Purpose

This document serves as a reference for both the production of the program as well as an easy entryway for outsiders to grasp the overall structure.

1.2 Scope

The program can return the output of a supplied code snippet, together with a rough estimate of its algorithmic complexity. However, in no way will the program be able to serve as an IDE, nor an advanced text editor. Therefore, debugging, version control or other complex features will not be implemented.

In scope:

- Sandboxed code execution
- Programming language selection
- Resource limiting
- Complexity analysis
 - Static analysis
 - Dynamic analysis
- Text input and output
- Complexity visualization/information display

Out of scope:

- Debugging tools (stepping, pausing, etc.)
- Code Storage
- Input file support

1.3 Tools and libraries

Tools/Frameworks

- React JS
- FastAPI
- Docker
- git
- gvisor (runsc)

Libraries

- sympy
- numpy
- tree-sitter
- etc.

1.4 Definitions and Acronyms

Term	Definition
Complexity	Time (not space) required to run an algorithm

1.5 References

- [Static time complexity analysis by Luzgan](#)
- [Dynamic time complexity analysis by pberkes](#)
- [blogs4devs Medium article RCE](#)
- [gvisor Guide](#)
- [Docker Docs](#)
- Goldsmith et al. "Measuring Empirical Computational Complexity" (FSE 2007)

2 System Overview

2.1 User Experience

The program is primarily for students or educators who can enter a code-snippet into a text window, select a supported programming language. On a window alongside it the users receive the programs output alongside a short breakdown of the algorithmic complexity.

2.2 Goals and Non-Goals

- **Goal:**
 - complexity estimate in "Big-O" Notation
 - analysis support recursion, standard-library calls, helper functions
 - Must be extendable for multiple programming languages
 - provide basic "secure" sandbox
 - User Kernel with gvisor
 - Resource limiting and Input size limiting
- **Non-goal:**
 - analysis does not support external Libraries
 - output besides text
 - Syntax highlighting
 - Production ready Sandbox (e.g. with tools like Firecracker)
 - allow user code to access networking

2.3 Assumptions and Constraints

- Total number of simultaneous users will be fairly limited
- Initially there will probably only be one language, but the structure should allow for multiple to be added later easily

- Sandbox Security might be lacluster -> focus on complexity analysis
- The complexity estimate will not be 100 % accurate
- The compleity analysis assumes function length to be rather small
- The scope of the project might be reduced to finish in a timely manner

3 Architecture

3.1 High-Level Architecture

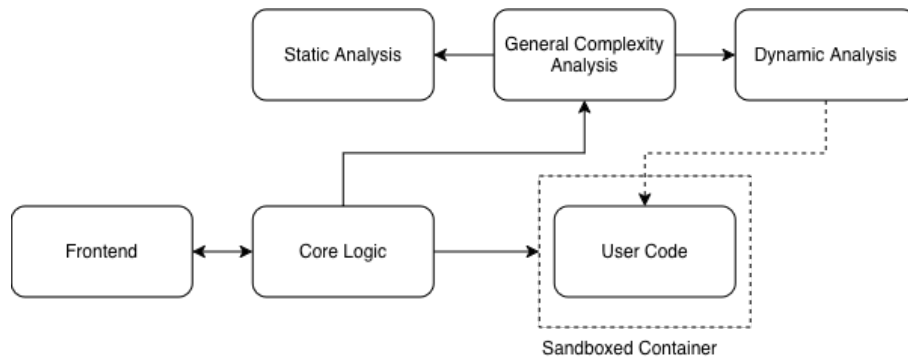


Figure 1: Architecture diagram

3.2 Component Descriptions

3.2.1 Frontend

This component handles the GUI and its logic. It also deals with the corresponding requests to the Core Logic. Additionally the styling of the webpage is also implemented here.

3.2.2 Core Logic

The Core Logic has two distinct responsibilities. Firstly, it handles the requests sent by the Frontend and formats the necessary data appropriately. Secondly, it manages the sandbox enviroment, instantiates new users with the appropriate programming language and oversees the resources consumed by each sandbox.

3.2.3 Sandboxed Container

The Sandboxed Container is responsible for compiling and executing the provided file with the user code.

3.2.4 General Complexity Analysis

Here the communication between the Static Analysis and Dynamic Analysis is combined for a full picture of the presumed algorithmic complexity. Alongside formatting the output and complexity data the module provides the response to the Core Logic. The two sub-modules are divided to allow for multiple different approaches depending on the users submitted code (i.e. stronger restrictions on the code -> better estimates) and to compare the two approaches independently of one another.

3.2.5 Static Analysis

Static Analysis means it does not oversee the execution of the code. It judges the codes complexity by evaluating the raw syntax and reports its conclusion back to the General Complexity Analysis module.

3.2.6 Dynamic Analysis

On the other hand, dynamic Analysis directly tracks the operations and speed of the executed code. It also reports its findings to the General Complexity Analysis module.

4 Detailed Design

4.1 Module: Core Logic

Responsibility: Owns the General Complexity Analysis module as well as the container running the users code.

Key interfaces:

```

1  @app.get("/result", )
2  async def get_result(self, session_token) -> dict, session_token:
3      #returns the current results

```

Listing 1: Acquiring output and complexity estimate

```

1  @app.post("/execute", )
2  async def execute_code(self, session_token, execute: dict) -> dict,
    session_token:
3      #Creates and then deletes container which executes code based
        on the selected programming language

```

Listing 2: Executing supplied code

Key Data Structures:

```

1  execute{
2      code: string,
3      input: string,
4      input_type: string,
5      language: string
6  }

```

Listing 3: execute data structure

```

1  result{
2      otuput: string,
3      complexity: complexity,
4      certainty: percentage
5      analysis_strength: string
6  }

```

Listing 4: result data structure

```

1  request={
2      session_token: string
3      execute: execute
4      status = string (queued, running, done, error)

```

```

5 }
6
7 queue = [request] (fixed length)
8 running = [request]
9 done = [request]
10 error = [request]

```

Listing 5: request queue

Processing Logic:

```

1 #Use docker-py to control docker via python
2 if json contains code:
3     switch depending on programming language
4     copy start block that reads sys.argv, measures time from inside the
5     container and calls main(n) to the end of the script{
6     import sys, time, json
7
8     # --- injected by the platform ---
9     if __name__ == "__main__":
10         n = json.load(sys.stdin)
11         # warm-up iteration (optional, to stabilise caches)
12         _ = main(n)
13
14         # actual timed run
15         t_start = time.perf_counter()
16         result = main(n)
17         t_end = time.perf_counter()
18         elapsed = t_end - t_start
19
20         # Output a single JSON line that the host can parse
21         print(json.dumps({
22             "result": result,
23             "elapsed": elapsed
24         }))
25     }
26
27     create tar archive with python script and input file
28     convert to bytes
29     create a tar entry for the script
30     add the script to the tar archive
31     add requirements.txt
32     create container with memorylimit, memswap limit, process limits,
33     cpuquotas, disable network, runsc, limit writable storage
34     copy tar archive to /app in container
35     pass docker container and execute variable to complexity analyser
36     then wait for possible errors from complexity analyser
37     if no errors:
38         format complexity analysers answer into results
39         delete docker container
40     else:
41         display errors in frontend

```

Listing 6: container creation logic in pseudocode

Error Handling: 1. Docker API errors: Docker ApiErrors should be caught in this module since it should be the only one creating new containers. 2. Container execution errors: Container execution errors, timeout errors etc. will be caught by the Complexity Analysis module, since it actually runs the scripts in the container.

Design decisions: The combination of Docker and gVisor is defensible for this usecase since the server hosting the site is not expected to store personal data and other isolation layers would be detrimental to finishing in a timely manner. The use of the tar archive is necessary, because the Dynamic Analysis module needs to be able to run the user-script multiple times, in quick succession and with different parameters.

4.2 Module: General Complexity Analysis

Responsibility: Manages the Dynamic and Static Analysis modules.

Key interfaces:

```

1 def start_static_analyser(self, code, input, input_type) -> float:
2     pass static info to the module,
3     if there are errors give them to the core logic module
4     else save the complexity estimate and confidency
5 def start_dynamic_analyser(self, container, input, input_type)-> float:
6     pass info to the module,
7     if there are errors give them to the core logic module
8     else save the complexity estimate and confidency

```

Listing 7: Static and Dynamic interface

Key Data Structures:

```

1 complexity{
2     family: string, #Constant/Logarithmic/Polynomial/Super-Polynomial
3     value: float,
4 }

```

Listing 8: execute data structure

Processing Logic:

```

1 try to container.run(image=image,command=["python", "app.py", str(n)],
2     detach=False,remove=False,stdout=True, stderr=True)
3 save stdout/result and time to output
4 try start dynamic analysis
5     catch execution errors:
6         raise execution error from dynamic analyser
7 start static analysis
8 if static analysis impossible:
9     return dynamicComplexity, n/t plotting, aproximated function and
10     output
11 else:
12     uncertainty = compare_complexity()
13     return dynamicComplexity, n/t plotting, aproximated function,
14     staticComplexity, uncertainty, output
15 catch execution errors:
16     raise execution error to core logic from container

```

Listing 9: Analyse full complexity

```

1 dynamicDegree, dynamicTheta = map_complexity(dynamicComplexity)
2 staticDegree, staticTheta = map_complexity(staticComplexity)
3 if dynamicDegree != infinity and staticDegree != infinity:
4     delta_degree = abs(dynamicDegree - staticDegree)
5     if delta_degree == 0:
6         uncertainty = absolute(dynamicTheta - staticTheta) * 0.1

```

```

7     return uncertainty
8 else:
9     #different polynomial degree
10    uncertainty = delta_degree + absolute(dynamicTheta - staticTheta) *
        0.1
11    return uncertainty
12
13 else if dynamicDegree == infinity and not staticDegree == infinity:
14     finite_degree = dynamicDegree if dynamicDegree != infinity else
        staticDegree
15     #difference between P and NP huge problem!
16     uncertainty = 90.0 + 10.0 / finite_degree
17     return uncertainty
18
19 else:
20     if dynamicTheta == infinity and staticTheta == infinity:
21         return 0.0 #factorial
22     if dynamicTheta == infinity or staticTheta == infinity
23         return 50.0 #exponential or factorial
24
25     uncertainty = absolute(dynamicTheta - staticTheta) * 5.0
26     return uncertainty

```

Listing 10: Complexity comparison

```

1 switch complexity.family.name:
2     case "constant":
3         degree = 0
4         theta = 0
5     case "logarithmic":
6         degree = 0
7         theta = complexity.value
8     case "polynomial":
9         degree = complexity.value
10        theta = 0
11    case "non-polynomial":
12        degree = infinity
13        theta = complexity.value
14    case default:
15        raise error
16    return degree, theta

```

Listing 11: map complexity

Design decisions: The confidency values are largely arbitrary (on a scale of 100), but since their usecase is only qualitative this is fine. We still return dynamic and static complexity even if they differ largely, this might overstate the accuracy of the estimates, but it is still useful for debugging both the program and the users code.

4.3 Module: Dynamic Complexity Analysis

Responsibility: Re-runs the script in the contrainer multiple times and reports back to the General Complexity Analysis module

Key interfaces:

Key Data Structures:

```

1 complexity{
2     family: string, #Constant/Logarithmic/Polynomial/Super-Polynomial
3     value: float,
4 }

```

Listing 12: execute data structure

Processing Logic:

```

1 if input_type != none:
2     n, time = execute_script_with_autogenerated_data()
3
4     if n == 0 or t == 0:
5         raise ValueError
6
7     if len(n) < 5:
8         raise warning (unreliability of data)
9
10    predictions = {}
11
12    # constant
13    parameters, covariance = curve_fit(constant_progression, n, t)
14    predictions.update({"constant": [parameters, covariance]})
15
16    # logarithmic
17    parameters, covariance = curve_fit(logarithmic_progression, n, t)
18    predictions.update({"logarithmic": [parameters, covariance]})
19
20    # polynomial
21    parameters, covariance = curve_fit(polynomial_progression, n, t)
22    predictions.update({"polynomial": [parameters, covariance]})
23
24    # exponential / super-polynomial
25    parameters, covariance = curve_fit(exponential_progression, n, t)
26    predictions.update({"exponential": [parameters, covariance]})
27
28    predictions = {}
29    #residual sum of squares approach
30    for name, model_func, k in [
31        ("constant", constant_progression, 1),
32        ("logarithmic", logarithmic_progression, 2),
33        ("polynomial", polynomial_progression, 3),
34        ("exponential", exponential_progression, 3),
35    ]:
36        params, _ = curve_fit(model_func, n, t)
37        y_pred = [model_func(ni, *params) for ni in n]
38        rss = sum((ti - yi) ** 2 for ti, yi in zip(t, y_pred))
39        aic = len(n) * log(rss / len(n)) + 2 * k
40        aicc = aic + (2 * k * (k + 1)) / (len(n) - k - 1)
41        predictions[name] = {"params": params, "rss": rss, "aicc": aicc}
42
43    best_model = min(predictions, key=lambda name: predictions[name]["aicc"])
44    return predictions, best_model, n, time

```

Listing 13: calculate dynamic complexity

```

1 def constant_progression(c):

```

```

2     return c
3
4 def logarithmic_progression(n, a, b)
5     return a * log(n) + b
6
7 def polynomial_progression(n, a, k, b):
8     return m * (n ** k) + b
9
10 def exponential_progression()
11     return a * exponent(b * n) + c

```

Listing 14: complexity functions

```

1 try:
2     switch input_type:
3         case "string":
4             values = [ ''.join(random.choices(string.ascii_letters + string.
5                 digits, k=int(math.pow(i, 4)))) for i in range(5)]
6             results = fullRun(values)
7         case "int":
8             values = [int(math.pow(i, 5)) for i in range(5)]
9             results = fullRun(values)
10        case "float":
11            values = [math.pow(i, 5) + random.random() for i in range(5)]
12            results = fullRun(values)
13        case "int []":
14            values = []
15            for i in range(5):
16                value = [j for j in range(int(math.pow(i, 3)))]
17                random.shuffle(value)
18                values.append(value)
19            results = fullRun(values)
20        case "string []":
21            values = []
22            for i in range(5):
23                value = []
24                for j in range(5):
25                    v = ( ''.join(random.choices(string.ascii_letters + string.
26                        digits, k=int(math.pow(j, 3))))
27                    value.append(v)
28                    random.shuffle(value)
29                    values.append(value)
30            results = fullRun(values)
31        case "int [] []":
32            values = []
33            for i in range(5):
34                value = []
35                for j in range i:
36                    v = []
37                    v.append(random.randrange(stop=i))
38                    value.append(v)
39                values.append(value)
40            results = fullRun(values)
41        case default:
42            raise ValueError
43    print (results)
44    return results
45 catch:

```

```
44 raise Execution Errors
```

Listing 15: execute script with autogenerated data

```
1 random.shuffle(values)
2 results = {}
3 for n in values
4     result, duration = medianRunScript(n)
5     results.update({n: duration})
6 return results
```

Listing 16: full run with supplied data

```
1 container.run("find every file not named app in /app and delete")
2 container.run("delete every file of type directory in /app")
3 container.run("delete everything in /tmp")
```

Listing 17: reset environment

```
1 values = []
2 for i in range(3):
3     reset_environment()
4     value, duration = container.run(
5         command=["python", "app.py", str(n)],
6         detach=False,
7         remove=False,
8         stdout=True, stderr=True
9     )
10    values.append(duration, value)
11
12 return sorted(values, key=lambda tuple: tuple[0])
```

Listing 18: median run script

Design decisions: Excluding $n * \log(n)$ and factorial complexity was done due to the inherent imprecision of the time-measuring approach of complexity analysis and additionally highlights its problems when it comes to rigor. A limited set of autogenerated data will be made available to more easily test the program by constraining user input. *MedianRunScript()* is being constrained to python since other language functionality can be added later on.

4.4 Module: Static Complexity Analysis

Key Data Structures:

```
1 function{
2     name: string,
3     parameters:
4     parent_function_or_class: string
5     body: string
6     called_functions: string
7     recursive_calls: [{}]
8     complexity: string
9 }
```

Listing 19: execute data structure

Processing Logic:

```

1 tree = tree-sitter-python.parse(code)
2 return tree

```

Listing 20: parse file to syntax tree

```

1 functions = extract_functions()
2 parentLessFunctions = []
3 for f in functions:
4     if f.parent == None:
5         parentLessFunctions.append(f)
6
7 complexity = ""
8 for f in parentLessFunctions:
9     complexity.append(f.complexity)
10
11 return complexity_to_BigO_Notation(complexity)

```

Listing 21: combine and format complexity

```

1 function_stack = []
2 functions = [{}]
3 tree-sitter.walkTree:
4     for node in tree:
5         name = node.getName().text
6         parent_function = function_stack[-1] if function_stack else None
7         function = {
8             "name": name,
9             "parent": parent_function,
10            "children": [],
11            "parameters": [],
12            "complexity": analyzeFunction(node, current_complexity =
                parent_function.complexity if parent_function else 1,
                loop_depth=0)
13            "body": node.child_by_field_name("body")
14        }
15        if parent_func:
16            for f in functions:
17                if f["name"] == parent_func:
18                    f["children"].append(func_name)
19    return functions

```

Listing 22: extract functions

```

1 recursiveCalls = getRecursiveCalls(function)
2 if recursiveCalls.length >= 1:
3     try:
4         a, b, k = calculate_recursive_variables(recursiveCalls)
5         current_complexity = apply_akra_bazzi(a, b, k, current_complexity)
6     catch:
7         raise Error (Recursive Structure does not fit AkraBazzi -> only
            dynamic approach will get applied)
8 elif node.type == "for_statement" or node.type == "while_statement":
9     loop_depth += 1
10    current_complexity *= f"n^{loop_depth}"
11 elif node.type == "if_statement":
12    current_complexity += " + 1" # Branches add, but don't multiply
13 for child in node.children:
14    analyze_complexity(child, current_complexity, loop_depth)

```

```
15 return current_complexity
```

Listing 23: analyze function

```
1 result = []
2     if node.type == "call":
3         callee = node.child_by_field_name("function")
4
5         if callee and source(callee) == function_name:
6             result.append(node)
7
8     for child in node.children:
9         result.extend(find_recursive_calls(child, function_name))
10
11 return result
```

Listing 24: get recursive calls

```
1 b = []
2 a = []
3 for call in recursiveCalls:
4     if call.binary_operator == "/":
5         if call.binary_operator.left.type == identifier and call.
6             binary_operator.right.type == integer:
7             b.append(1/call.binary_operator.right)
8         elif call.binary_operator.left.type == binary_operator and call.
9             binary_operator.right.type == integer:
10            if call.binary_operator.left == "*":
11                if call.binary_operator.left.left.type == integer:
12                    b.append(call.binary_operator.left.left/call.binary_operator.
13                        right)
14                elif call.binary_operator.left.right.type == integer:
15                    b.append(call.binary_operator.left.right/call.binary_operator
16                        .right)
17            else:
18                raise exception
19        else:
20            b.append(1/call.binary_operator.right)
21        else:
22            raise exception
23    elif call.binary_operator == "//":
24        if call.binary_operator.left.type == identifier and call.
25            binary_operator.right.type == integer:
26            b.append(1/call.binary_operator.right)
27        elif call.binary_operator.left.type == binary_operator and call.
28            binary_operator.right.type == integer:
29            if call.binary_operator.left.left.type == integer:
30                b.append(call.binary_operator.left.left/call.binary_operator.
31                    right)
32            elif call.binary_operator.left.right.type == integer:
33                b.append(call.binary_operator.left.right/call.binary_operator
34                    .right)
35            else:
36                raise exception
37        else:
38            raise exception
39    for shrinkageFactor in b:
40        a.append(b.count(shrinkageFactor))
```

```

33
34 return a, b, b.length()

```

Listing 25: calculate recursive Variables

```

1 expression = string_to_sympy(current_complexity)
2 u = symbols('u', positive=True)
3 p = compute_p(a, b)
4
5 integral = expand(expression.subs(x, u) / u**(p + 1))
6 F = integrate(integral, u)
7 X = symbols('X', positive=True)
8 definite = F.subs(u, X) - F.subs(u, 1)
9
10 T_expr = X**p * (1 + definite)
11 simplify(T_expr)
12
13 return convert_sympy_to_string(simplify(T_expr))

```

Listing 26: apply akra bazzi

```

1 """
2 Returns the unique real p such that sum(a_i * b_i^p) == 1.
3 """
4 # Expand search range if needed
5 def f(p):
6     return sum(a * (b ** p) for a, b in zip(a_list, b_list)) - 1.0
7
8 # Make sure the root is bracketed
9 while f(low) <= 0:
10     low *= 2.0
11 while f(high) >= 0:
12     high *= 2.0
13
14 # Binary search (the function is monotonic decreasing)
15 for _ in range(200): # enough for ~1e-60 precision
16     mid = (low + high) / 2.0
17     if f(mid) > 0:
18         low = mid
19     else:
20         high = mid
21 return (low + high) / 2.0

```

Listing 27: compute p

Design decisions: It needs to be acknowledged that the static analyser is in no way optimal. It doesn't handle library calls, mutual recursion or more complex programming patterns. For keeping the scope of the program in a manageable size those constraints were deemed necessary. The module retains relevancy since for simpler algorithms it provides an easily verifiable output to compare the dynamic analysis module to. Not using h_i in the application of the Akra-Bazzi Theorem was done due to its limited influence on the asymptotic behaviour.

5 Testing Strategy

- **Unit tests:** (limited scope)
 - Api calls put and get requests

- Getting the correct container output
- **Manual testing:**
 - Independent complexity results for known (and simple algorithms)
 - Complexity Averaging
 - Akra bazzi input parameters
 - dynamic function fitting (verify with displaying a graph and points)
- **Known edge cases to verify:**
 - recursion in static analyser
 - $n * \log(n)$ time in dynamic analyser
 - attempted library and system calls

6 Risks and Open Questions

- complexity formatting for analysis modules -> current idea is string formatting and simplifying
- narrowness of static analyser
- dynamic analysis says little about actual asymptotic behaviour -> still useful for real-world applications?
- Docker + gVisor is reasonable for a student project but can still be exploited -> no personal or sensitive data should be stored on the same server